

JSON and Byte File I/O

My teaching notes

A word before you read. These are my notes from over the years, cleaned up so they are readable. A few things to keep in mind:

- **They are not a study guide, and they are not gospel.** Not everything here appears in a lecture, and not every topic has notes yet; I will write more as we go.
- **I write from memory and revise every semester.** A typo or an imprecise statement can survive longer than it should.
- **I use AI to help** convert documents to markdown, check for issues, and add explanations. I do not trust it completely, so I am still editing these myself.
- **Java changes.** The basics we cover are pretty resilient, but some features do move over time.

So: if you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. When in doubt, the textbook is the tiebreaker, and the compiler beats us both.

These notes cover both days this week: reading and writing JSON files with the `org.json` library, and reading and writing binary files with `DataOutputStream` and `DataInputStream`. Every listing below compiles as written and has been run to confirm its output. Examples use a small **CrewMember** class, holding data for one crew member at a fictional outpost, so the same object shows up throughout, first stored as JSON, later stored as raw bytes.

1. Why Two New File Formats

Week 3 covered three ways to read text: a space-separated file, a CSV file split by hand, and a CSV file read with a library, plus one way to write text back out with `PrintWriter`. All of that stays text: every value on disk is stored as readable characters, whether that is a word, a number written as digits, or a comma between fields.

This week introduces two formats that are not just plain, unstructured text. JSON is still text, but it is structured text: every value has a name, and values can be grouped and nested, which a bare CSV line cannot do on its own. Binary is the opposite move: instead of writing a number as the characters that spell it out, a binary file stores the number's actual bytes, which is compact but no longer readable by eye.

2. JSON Basics: `JSONObject` and `JSONArray`

The `org.json` library represents a JSON object as a `JSONObject`, which behaves a lot like a map: a value goes in under a key with `put`, and comes back out later by that same key. A JSON array, an ordered list of values, is a `JSONArray`. Nesting a `JSONArray` inside a `JSONObject` is how a single field holds a list, for example one crew member's list of skills.

```
JSONObject crew = new JSONObject();
crew.put("name", "Priya Nair");
crew.put("role", "Engineer");
crew.put("health", 92);
crew.put("morale", 88.5);
crew.put("onDuty", true);

JSONArray skills = new JSONArray();
skills.put("Reactor Maintenance");
skills.put("Spacewalk Certified");
crew.put("skills", skills);
```

Calling `crew.toString()` gives the compact form, all on one line, the form that actually gets written to a file. Calling `crew.toString(2)` pretty-prints with two spaces per level of nesting, which is easier to read on screen but is not required by the JSON format itself:

```
{
  "skills": [
    "Reactor Maintenance",
    "Spacewalk Certified"
  ],
  "role": "Engineer",
  "name": "Priya Nair",
  "health": 92,
  "morale": 88.5,
  "onDuty": true
}
```

A note on key order. The keys above do not print in the order they were added. A `JSONObject` does not promise to remember insertion order, since a JSON object is unordered by definition; only a `JSONArray` promises to keep its items in order. This is normal and does not affect correctness.

Reading values back out uses a typed getter for each field: `getString`, `getInt`, `getDouble`, `getBoolean`, and `getJSONArray` for a nested array. Using the wrong getter, for example calling `getInt` on a field that is really a `String`, throws a `JSONException`.

```
String name = crew.getString("name");
int health = crew.getInt("health");
JSONArray skills = crew.getJSONArray("skills");
for (int i = 0; i < skills.length(); i++) {
    System.out.println(skills.getString(i));
}
```

3. Reading and Writing JSON Files

A `JSONObject` does not write itself to disk. Writing one to a file uses the same `FileWriter` we already know from Week 3, just writing the JSON text instead of our own formatted lines:

```
try (FileWriter writer = new FileWriter("crew.json")) {
```

```

        writer.write(crew.toString(2));
    } catch (IOException e) {
        System.out.println("Could not write the file: " + e.getMessage());
    }
}

```

Reading a JSON file back in is a two-step process: read the whole file into a single String, then build a JSONObject or JSONArray from that String. `Files.readString` reads an entire file in one call, which is convenient here since a JSONObject needs its full text at once, unlike Scanner reading a CSV file one line at a time.

```

String text = Files.readString(Path.of("crew.json"));
JSONObject crew = new JSONObject(text);

```

4. Converting Between a Custom Class and JSON

In a real program, data usually lives in a custom class, not directly in a JSONObject. Two small helper methods handle the conversion in each direction: one that builds a JSONObject from a CrewMember, and one that builds a CrewMember from a JSONObject.

```

private static JSONObject toJson(CrewMember member) {
    JSONObject obj = new JSONObject();
    obj.put("name", member.getName());
    obj.put("role", member.getRole());
    obj.put("health", member.getHealth());
    obj.put("morale", member.getMorale());
    obj.put("onDuty", member.isOnDuty());
    obj.put("skills", new JSONArray(member.getSkills()));
    return obj;
}

private static CrewMember fromJson(JSONObject obj) {
    CrewMember member = new CrewMember(
        obj.getString("name"),
        obj.getString("role"),
        obj.getInt("health"),
        obj.getDouble("morale"),
        obj.getBoolean("onDuty"));
    JSONArray skills = obj.getJSONArray("skills");
    for (int i = 0; i < skills.length(); i++) {
        member.addSkill(skills.getString(i));
    }
    return member;
}

```

A whole roster is then just a JSONArray of JSONObjects: loop over an `ArrayList<CrewMember>` calling `toJson` on each one and adding the result to a JSONArray to write the roster, or loop over a JSONArray calling `fromJson` on each entry to rebuild the `ArrayList` when reading it back.

Table 1: Text, JSON, and binary files compared

	Plain text / CSV	JSON	Binary
Human-readable	Yes	Yes	No
Self-describing (named fields)	No	Yes	No
Supports nested structure	No	Yes	With planning
Typically more compact	No	No	Yes
Reading requires exact order	Often	No	Yes

5. Comparing File Formats

None of these formats is simply better than the others. CSV is compact and simple for flat, table-shaped data. JSON trades some size and speed for being self-describing and able to represent nested structure. Binary trades readability for compactness and speed, which matters more for very large files or files a program reads and writes often, such as a saved game state.

6. Byte Streams: Raw Bytes and Typed Primitives

A binary file is written with a different family of classes than a text file. `FileOutputStream` and `FileInputStream` move raw bytes, one at a time or in blocks, with no idea of what those bytes mean. `DataOutputStream` and `DataInputStream` wrap around them to add methods for writing and reading Java's primitive types directly as bytes: `writeInt`, `writeDouble`, `writeBoolean`, and `writeUTF` for a `String`.

```
try (DataOutputStream out = new DataOutputStream(new FileOutputStream("save.dat"))) {
    out.writeUTF("Priya Nair");    // name
    out.writeUTF("Engineer");     // role
    out.writeInt(92);             // health
    out.writeDouble(88.5);        // morale
    out.writeBoolean(true);       // onDuty
} catch (IOException e) {
    System.out.println("Could not write the file: " + e.getMessage());
}
```

Reading the file back in requires calling the matching `readX` methods in the exact same order the data was written. Nothing in the file records that order; the program has to already know it, unlike JSON, where each value is found by name instead of by position.

```
try (DataInputStream in = new DataInputStream(new FileInputStream("save.dat"))) {
    String name = in.readUTF();    // must match the writeUTF order
    String role = in.readUTF();
    int health = in.readInt();
    double morale = in.readDouble();
    boolean onDuty = in.readBoolean();
} catch (IOException e) {
    System.out.println("Could not read the file: " + e.getMessage());
}
```

A note on order. If the readX calls are in the wrong order, or the wrong type, the program will not throw a clean error every time. It may instead read back nonsense values, since a `DataInputStream` has no way to check that the bytes it is decoding as, say, an int were actually written as an int. Getting the order right is entirely the programmer's responsibility.

Why a binary file is not human-readable

Opening `save.dat` in a plain text editor does not show anything close to the file's contents. A text editor treats every byte as a character to display, and most of the byte values a `DataOutputStream` writes do not correspond to a printable character at all. Looking at the raw bytes directly makes this clear:

```
0 10 80 114 105 121 97 32 78 97 105 114 0 8 69 110 103 105 110 101 101 114 0 0
```

There are no quotes, no field names, and no readable structure in that output, only numbers. For the same crew member's data, `save.dat` came to 81 bytes, compared to 172 bytes for the equivalent `crew.json`. Binary saves space by dropping everything that made the JSON version readable: the field names, the quotes, and the indentation.

7. Storing a List in a Binary File

A binary file has no natural way to mark where a list ends the way a `JSONArray`'s closing bracket does. The standard fix is to write a count before the items themselves, so a reader knows exactly how many items to read back. The same idea applies at two levels at once for a roster: once for the whole roster, and once again for each crew member's own list of skills.

```
private static void writeCrewMember(DataOutputStream out, CrewMember member)
    throws IOException {
    out.writeUTF(member.getName());
    out.writeUTF(member.getRole());
    out.writeInt(member.getHealth());
    out.writeDouble(member.getMorale());
    out.writeBoolean(member.isOnDuty());

    ArrayList<String> skills = member.getSkills();
    out.writeInt(skills.size());           // count first
    for (String skill : skills) {
        out.writeUTF(skill);
    }
}
```

Writing a whole roster wraps this same method in a loop, after first writing the roster's own size:

```
out.writeInt(roster.size());           // how many records follow
for (CrewMember member : roster) {
    writeCrewMember(out, member);
}
```

Reading it back mirrors the write exactly: read the roster's size first, then loop that many times,

and inside each iteration, read one crew member's fields followed by its own skill count and that many skills.

8. Putting It Together

Both days this week move the same kind of data, a `CrewMember`, in and out of a file. Only the format changes: named, nested JSON on day one, ordered raw bytes on day two. Comparing the two side by side is the fastest way to see what each format is actually trading off. JSON costs some size and requires a library, but stays readable and self-describing. Binary is compact and fast, but demands that the program remember the exact order and type of everything it wrote, with no help from the file itself.